Writestring

Summary

The eyeon.writestring will write strings and values as a string into a metadata table. This function serializes a table into a string. It is also possible to create a table with the proper structure from scratch.

Usage

eyeon.writestring(*string* val)

val (*string*, required)

DCTL Processing

DCTL Introduction

DCTL is an internal language that is processed on the GPU of the host computer and is abstracted to compile to different GPU processes, like Metal on OSX, OpenCL for AMD GPUs and CUDA for Nvidia GPUs.

The DCTL syntax is C-like with additional definitions. Users can define functions using DCTL code to create a video effect and run it via Fuses in Resolve and Fusion. DCTL Kernels serve as a "pixel shader" program, a process to generate one pixel of data at a time at each given frame's coordinates.

DCTL effects can be run as a Fuse Tool Plugin in a comp. These typically deal with a 2D texture image of RGBA float values, or of alpha values only.

The basic programing concept is similar to the GLSL shader kernels with different syntax; first define parameters to pass values from the fuse to the Kernel, second is to define the Kernel that does the process on textures.

Kernels

DCTL Parameters

Parameters are defined in structure and contain integer, float, and array values. Any number and type of parameters can be defined. The Parameter structure is a user-defined string, typically contained in the raw string delimiters of double square brackets.

NameParams = [[...]]

Examples

```
GradientParams = [[
    float col[4];
    int dstsize[2];
]]

CircleParams = [[
    float amp;
    float damp;
    float freq;
    float phase;
    float center[2];
    int srcsize[2];
    int compOrder;
]]
```

DCTL Kernel Source Code

The Kernel is the image processing code that will be executed when called. The Kernel source code is a user-defined string, and is typically contained in the raw string delimiters of double square brackets.

KernelSource = [[...Source Code...]]

Kernel functions that can be called from the fuse using a compute node are defined by the __KERNEL__ prefix, with double underscores.

```
GradientSource = [[
__KERNEL__ void GradientKernel(__CONSTANTREF__ GradientParams *params,
    __TEXTURE2D_WRITE__ dst)
{
    DEFINE_KERNEL_ITERATORS_XY(x, y)
    if (x < params->dstsize[0] && y < params->dstsize[1])
    {
           float2 pos = to_float2(x, y) / to_float2(params->dstsize[0] -
           1, params->dstsize[1] - 1);

           float4 col = to_float4_v(params->col);
           col *= to_float4(pos.x, pos.y, 0.0f, 1.0f);

           _tex2DVec4Write(dst, x, y, col);  // image, x, y, float4 colour
    }
}
]]
```

__KERNEL__

Summary

This defines the entry to the Kernel code and name of the structure passing in parameters. The pointer to the image texture is also defined.

Usage

__KERNEL__ void KernelName(__CONSTANTREF__ ParamsName *params, __TEXTURE2D__ inputTexture, __TEXTURE2D_WRITE__ outputTexture)

KernelName

A user defined reference name of the Kernel to be called from the process.

ParamsName

The user-defined reference name of the parameter structure containing values into the kernel.

Qualifers

These qualifiers are used:

__CONSTANT__ qualifier to define fast constant memory.

__CONSTANTREF__ qualifier for a constant memory parameter structure passed to a function.

Textures

The textures to be passed to the kernel for processing. The type of texture is defined by the following qualifiers

Qualifiers

__TEXTURE2D__ Read-only two-dimensional texture.

__TEXTURE2D_WRITE__ 2D texture that the kernel can write into.

Math Functions

```
float _fabs(float x)
```
Returns the absolute value of x

```
float _powf(float x, float y)
```
Computes x raised to the power of y

```
float _logf(float x)
```
Computes the value of the natural logarithm of x

```
float _log2f(float x)
```
Computes the value of the logarithm of x to base 2

```
float _log10f(float x)
```
Computes the value of the logarithm of x to base 10

```
float _expf(float x)
```
Computes e**x, the base-e exponential of x

```
float _exp2f(float x)
```
Computes 2**x, the base-2 exponential of x

```
float _exp10f(float x)
```
Computes 10**x, the base-10 exponential of x

```
float _copysignf(float x, float y)
```
Returns x with its sign changed to y's

```
float _fmaxf(float x, float y)
```
Returns x or y, whichever is larger

```
float _fminf(float x, float y)
```
Returns x or y, whichever is smaller

```
float _clampf(float x, float min, float max)
```
Clamps x to be within the interval [min, max]

```
float _saturatef(float x)
```
Clamps x to be within the interval [0.0f, 1.0f]

```
float _sqrtf(float x)
```
Computes the non-negative square root of x

```
float _ceilf(float x)
```
Returns the smallest integral value greater than or equal to x

```
float _floorf(float x)
```
Returns the largest integral value less than or equal to x

```
float _truncf(float x)
```
Returns the integral value nearest to but no larger in magnitude than x

```
float _round(float x)
```
Returns the integral value nearest to x rounding, with half-way cases rounded away from zero

```
float _fmod(float x, float y)
```
Computes the floating-point remainder of x/y

```
float _hypotf(float x, float y)
```
Computes the square root of the sum of squares of x and y

```
float _cosf(float x)
```
Computes the cosine of x (measured in radians)

```
float _sinf(float x)
```
Computes the sine of x (measured in radians)

```
float _cospif(float x)
```
Computes the cosine of (x * pi) (measured in radians)

```
float _sinpif(float x)
```
Computes the sine of (x * pi) (measured in radians)

```
float _tanf(float x)
```
Computes the tangent of x (measured in radians)

```
float _acosf(float x)
```
Computes the principal value of the arc cosine of x

```
float _asinf(float x)
```
Computes the principal value of the arc sine of x

```
float _atan2f(float y, float x)
```
Computes the principal value of the arc tangent of y/x, using the signs of both arguments to determine the quadrant of the return value

```
float _acoshf(float x)
```
Computes the principal value of the inverse hyperbolic cosine of x

```
float _asinhf(float x)
```
Computes the principal value of the inverse hyperbolic sine of x

```
float _atanhf(float x)
```
Computes the inverse hyperbolic tangent of x

```
float _coshf(float x)
```
Computes the hyperbolic cosine of x

```
float _sinhf(float x)
```
Computes the hyperbolic sine of x

```
float _tanhf(float x)
```
Computes the hyperbolic tangent of x

```
float _fdimf(float x, float y)
```
Returns the positive difference between x and y:x - y if x > y, +0 if x is less than or equal to y

```
float _fmaf(float x, float y, float z)
```
Computes (x * y) + z as a single operation

```
float _rsqrtf(float x)
```
Computes the reciprocal of square root of x

```
float _fdivide(float x, float y)
```
Returns x/y

```
float _frecip(float x)
```
Returns 1/x

```
int isinf(float x)
```
Returns a non-zero value if and only if x is an infinite value

```
int isnan(float x)
```
Returns a non-zero value if and only if x is a NaN value

```
int signbit(float x)
```
Returns a non-zero value if and only if sign bit of x is set

```
T _mix(T x, T y, float a)
```
T is used to indicate that the function can take float, float2, float3, float4, as the type for the arguments.Returns: (x + (y - x) * a). "a" must be a value in the range [0.0f, 1.0f]. If not, the return values are undefined.

```
float _frexp(float x, int exp)
```
Extracts mantissa and exponent from x. The mantissa m returned is a float with magnitude in the interval [1/2, 1) or 0, and exp is updated with integer exponent value, whereas x = m * 2^exp

```
float _ldexp(float x, int exp)
```
Returns (x * 2^exp)

NOTE that float values must have 'f' character at the end (e.g. 1.2f).

List of integer math functions available

```
int abs(int x)
```
Returns the absolute value of x

```
int min(int x, int y)
```
Returns x or y, whichever is smaller

```
int max(int x, int y)
```
Returns x or y, whichever is larger

define_kernel_iterators_xy(x, y)

Summary

This sets the x and y variables to the integer coordinates of the pixel currently being processed, and will typically vary between (0,0) and the (width,height) of the output texture being written to. Note that the GPU kernel may sometimes round the maximum values up for performance reasons, so you should be careful not to attempt writing to (x,y) coordinates greater than (width-1,height-1) of your output texture.

_tex2DVec4Write

Summary

Writes a four-channel RGBA color to a specific pixel in the output texture.

Usage

_tex2DVec4Write(*tex, x, y, color*);

tex	Pointer to output texture
x	location of pixel in the X axis
y	location of pixel in the Y axis
color	float4 vector of RGBA color.

_tex2DVec4(tex, x, y)

_tex2DVecN(tex, x, y, order)

Summary

_tex2DVec4(tex, x, y) and _tex2DVecN(tex, x, y, order) are the default image sampling functions. They read a float4 vector of RGBA color values from a given texture tex at the specified x and y coordinates, which are specified in pixels by default, i.e. between (0,0) and (width-1,height-1) inclusive. The coordinate space, pixel filtering, and edge behavior can be changed using the AddSampler() function with the default sampler name RowSampler.

_tex2DVecN() also takes an order parameter, which is a bitmask describing which channels to read. Pass 15 to return all four RGBA channels, or pass 1 to read just the alpha value and return it in all four RGBA channels.

_tex2DSamplerVec4(tex, sampler, x, y)

_tex2DSamplerVecN(tex, sampler, x, y, order)

Summary

_tex2DSamplerVec4(tex, sampler, x, y) and _tex2DSamplerVecN(tex, sampler, x, y, order) read a float4 vector of RGBA color values from a given texture tex at the specified x and y coordinates, using the coordinate space, pixel filtering, and edge behavior modes given to an additional sampler, which is set up with the AddSampler() function.

_tex2DSamplerVecN() also takes an order parameter, which is a bitmask describing which channels to read. Pass 15 to return all four RGBA channels, or pass 1 to read just the alpha value and return it in all four RGBA channels.

User Defined Functions

Summary

User defined functions can also be created and called.

Usage

__DEVICE__ float *functionname*(float value, float *value*.....) {*return*}

Example

```
__DEVICE__ float luma( float r, float g, float b ) {
    return ((0.2126 * r) + (0.7152 * g) + (0.0722 * b));
}
```

Process

Process Introduction

With the DCTL Kernel and parameter structures defined, the fuse's processing function follows the same lines as other Fuse tools. Images and UI elements like sliders and on screen controls will be requested for values, which are aggregated in the Request object given to the Process() function, and can then be copied into a block of parameters to be passed to the GPU compute kernel for processing.

DVIPComputeNode

Summary

This creates a compute node for processing image pixels with a GPU compute kernel.The kernel is compiled from a DCTL source string, and is given a block of parameter values to use.

Usage

nodename = DVIPComputeNode(req, `"KernelSource"`, *KernelSource*, `"NameParams"`, *ParamBlock*)

nodename	Is a reference to the compute node.
req	Is the Request object passed to the fuse's Process() function.
KernelSource	Is a reference to the defined kernel source code string.
NameParams	Is the name of the parameter structure as used by the kernel source.
ParamBlock	Is a reference to the block of parameter values that will be passed to the kernel.

Example

```
local node = DVIPComputeNode(req, "GradientKernel", GradientSource,
"GradientParams", GradientParams)
```

GetParamBlock

Summary

This gets a reference to the block of parameter values that will be passed as a structure to the compute kernel.

Usage

parameterRef = *node*:GetParamBlock(*ParamBlock*)

parameterRef	Is a reference to the parameter block
node	Is a reference to the kernel compute node
ParamBlock	Is a reference to the parameter structure that will pass values to the kernel

Example

```
local params = node:GetParamBlock(CircleParams)

params.amp = amp
params.damp = damp
params.freq = freq
params.phase = phase
```

SetParamBlock

Summary

This will pass the values from the parameter block to the kernel.

Usage

node:SetParamBlock(*ParamBlock*)

node	Is a reference to the compute node
ParamBlock	Is a reference to the block of parameter values to be given to the kernel

Example

```
local params = node:GetParamBlock(CircleParams)

params.center[0] = center.X
params.center[1] = center.Y
params.compOrder = 15
params.srcsize[0] = out.DataWindow:Width()
params.srcsize[1] = out.DataWindow:Height()

node:SetParamBlock(params)
```

AddInput

Summary

This creates a texture from an Image, and sends it to the kernel for reading from. Each call to AddInput() must have a matching argument in the DCTL kernel function of the form **__TEXTURE2D__ name**. A kernel may take multiple input textures, or none at all.

Usage

node:AddInput(*name*, *image*)

node	Is a reference to the kernel compute node
name	Is a string that identifies the texture passed to the kernel
image	Is a reference to an Image object

Example

```
node:AddInput("src", img)
```

AddOutput

Summary

This attaches an empty texture to an Image, and passes it to the kernel to be written to. A single call to AddOutput() is required, and it must have a matching argument in the DCTL kernel function of the form **__TEXTURE2D_WRITE__ name**.

Usage

node:AddInput(*name*, *image*)

node	Is a reference to the kernel compute node
name	Is a string that identifies the texture passed to the kernel
image	Is a reference to an Image object

Example

```
node:AddInput("src", img)
```

AddSampler

Summary

This tells the GPU how to find and read pixels from an image texture. There are a number of filter methods, edge condition treatments and coordinate modes.

Usage

AddSampler("*Sampler*", *filterMode*, *addressMode*, *normCoordsMode*)

RowSampler	**is the default image sampler. It has default settings of :** TEX_FILTER_MODE_POINT, TEX_ADDRESS_MODE_CLAMP, TEX_NORMALIZED_COORDS_FALSE
filterMode	TEX_FILTER_MODE_POINT (nearest sampling) TEX_FILTER_MODE_LINEAR (bilinear sampling)
addressMode	TEX_ADDRESS_MODE_BORDER (black edges) TEX_ADDRESS_MODE_CLAMP (duplicate edges) TEX_ADDRESS_MODE_MIRROR (mirrored edges) TEX_ADDRESS_MODE_WRAP (wrapped edges)
normCoordsMode	TEX_NORMALIZED_COORDS_FALSE (pixel coords, 0..width-1 and 0..height-1) TEX_NORMALIZED_COORDS_TRUE (normalised coords, 0..1) TEX_NORMALIZED_COORDS_AUTO (normalised coords for linear-sampled textures, otherwise pixel coords)

Example

```
    node:AddSampler("RowSampler", TEX_FILTER_MODE_LINEAR,
                TEX_ADDRESS_MODE_CLAMP, TEX_NORMALIZED_COORDS_TRUE)
```

RunSession

Summary

Once the compute node has been created, image textures created, and all values set in the parameter block, RunSession will execute the defined kernel on the GPU. A boolean value is returned to indicate the kernel has been successfully queued, or if a failure has occurred at some stage (such as a kernel compilation error). Any error conditions will be logged to the Console.

Usage

node:RunSession(*req*)

node	Is a reference to the compute node
req	Is the Request object passed to the kernel process.

Examples

```
    node:AddInput("src", img)
    node:AddOutput("dst", out)

    success = node:RunSession(req)
```

```
-- Registry declaration

FuRegisterClass("GPUSampleFuse", CT_SourceTool, {
    REGS_Category = "Fuses\\Examples",
    REGS_OpIconString = "GFu",
    REGS_OpDescription = "GPU Sample Fuse",

    REG_NoObjMatCtrls = true,
    REG_NoMotionBlurCtrls = true,

    REG_Source_GlobalCtrls = true,
    REG_Source_SizeCtrls = true,
    REG_Source_AspectCtrls = true,
    REG_Source_DepthCtrls = true,
    })
```

```
--------------------------------
-- Description of kernel parameters

GradientParams = [[
    float col[4];
    int dstsize[2];
]]

CircleParams = [[
    float amp;
    float damp;
    float freq;
    float phase;
    float center[2];
    int srcsize[2];
    int compOrder;
]]

--------------------------------
-- source of kernel


GradientSource = [[
__KERNEL__ void GradientKernel(__CONSTANTREF__ GradientParams *params,
    __TEXTURE2D_WRITE__ dst)
{
    DEFINE_KERNEL_ITERATORS_XY(x, y)
    if (x < params->dstsize[0] && y < params->dstsize[1])
    {
           float2 pos = to_float2(x, y) / to_float2(params->dstsize[0] - 1,
               params->dstsize[1] - 1);

           float4 col = to_float4_v(params->col);
           col *= to_float4(pos.x, pos.y, 0.0f, 1.0f);

           _tex2DVec4Write(dst, x, y, col);   // image, x, y, float4 color
    }
}
]]

CircleSource = [[
#define _length(a,b) _sqrtf(((a).x-(b).x)*((a).x-(b).x) + ((a).y-(b).y)*((a).y-
(b).y))

__KERNEL__ void CircleKernel(__CONSTANTREF__ CircleParams *params,
     __TEXTURE2D__ src, __TEXTURE2D_WRITE__ dst)
{
```

```
    DEFINE_KERNEL_ITERATORS_XY(x, y)
    if (x < params->srcsize[0] && y < params->srcsize[1])
    {
           float2 pos = to_float2(x, y) / to_float2(params->srcsize[0] - 1,
              params->srcsize[1] - 1);

           float2 center = to_float2_v(params->center);
           float d = _length(pos, center);
           float vl = fmax(params->amp - params->damp * d, 0.0f);
           vl = 1.0f + sin(d * params->freq + params->phase) * vl;

           float2 frompos = vl * (pos - center) + center;

              // source image, x & y coords, component mask
           float4 col = _tex2DVecN(src, frompos.x, frompos.y, params-
           >compOrder);_tex2DVec4Write(dst, x, y, col);
    }
}
]]


function Create()
    InCenter = self:AddInput("Center", "Center", {
           LINKID_DataType = "Point",
           INPID_InputControl = "OffsetControl",
           INPID_PreviewControl = "CrosshairControl",
           })

    InAmplitude = self:AddInput("Amplitude", "Amplitude", {
           LINKID_DataType = "Number",
           INPID_InputControl = "SliderControl",
           INP_Default = 0.5,
           })

    InDamping = self:AddInput("Damping", "Damping", {
           LINKID_DataType = "Number",
           INPID_InputControl = "SliderControl",
           INP_Default = 0.0,
           })

    InFrequency = self:AddInput("Frequency", "Frequency", {
           LINKID_DataType = "Number",
           INPID_InputControl = "SliderControl",
           INP_Default = 20,
           INP_MaxScale = 100.0,
           })
```

```
    InPhase = self:AddInput("Phase", "Phase", {
            LINKID_DataType = "Number",
            INPID_InputControl = "ScrewControl",
            INP_Default = 0.0,
            INP_MaxScale = 10.0,
            })

    -- OutImage is automatically created for us, as we're a CT_SourceTool
end

function Process(req)
    local center  = InCenter:GetValue(req)
    local amp = InAmplitude:GetValue(req).Value
    local damp = InDamping:GetValue(req).Value
    local freq = InFrequency:GetValue(req).Value
    local phase = InPhase:GetValue(req).Value

    local realwidth = Width;
    local realheight = Height;

    -- We'll handle proxy ourselves
    Width = Width / Scale
    Height = Height / Scale
    Scale = 1

    local imgattrs = {
            IMG_Document = self.Comp,
            IMG_Width = Width,
            IMG_Height = Height,
            IMG_XScale = XAspect,
            IMG_YScale = YAspect,
            IMAT_OriginalWidth = realwidth,
            IMAT_OriginalHeight = realheight,
            IMG_Quality = not req:IsQuick(),
            IMG_MotionBlurQuality = not req:IsNoMotionBlur(),
            }

    if not req:IsStampOnly() then
            imgattrs.IMG_ProxyScale = 1
    end

    if SourceDepth ~= 0 then
            imgattrs.IMG_Depth = SourceDepth
    end
```

```
    local img = Image(imgattrs)
    local out
    local success = false
    if img then
          local node = DVIPComputeNode(req, "GradientKernel",
GradientSource, "GradientParams", GradientParams)

          if node then
                -- create image
                local params = node:GetParamBlock(GradientParams)

                params.col[0] = 1.0
                params.col[1] = 1.0
                params.col[2] = 1.0
                params.col[3] = 1.0
                params.dstsize[0] = img.DataWindow:Width()
                params.dstsize[1] = img.DataWindow:Height()

                node:SetParamBlock(params)

                node:AddOutput("dst", img)

                success = node:RunSession(req)
          end

          -- and warp it
          if success then
                out = Image({IMG_Like = img })

                local node = DVIPComputeNode(req, "CircleKernel",
CircleSource, “CircleParams",   CircleParams)

                if node then
                      -- create image
                      local params = node:GetParamBlock(CircleParams)

                      params.amp = amp
                      params.damp = damp
                      params.freq = freq
                      params.phase = phase
                      params.center[0] = center.X
                      params.center[1] = center.Y
                      params.compOrder = 15
                      params.srcsize[0] = out.DataWindow:Width()
                      params.srcsize[1] = out.DataWindow:Height()
```

```
                        node:SetParamBlock(params)
                        node:AddSampler("RowSampler", TEX_FILTER_MODE_
                        LINEAR, TEX_ADDRESS_MODE_CLAMP, TEX_NORMALIZED_
                        COORDS_TRUE)
                        node:AddInput("src", img)
                        node:AddOutput("dst", out)

                        success = node:RunSession(req)
                else
                        out = nil
                end
        end
    end

    OutImage:Set(req, out)
end
```